

Gün 4 — EOF Ötesi Bayt Tarama ve Video Header Tespiti

Hedef

Gün 3'te üretilen polyglot dosyalarda, görselin gerçek bitiş imzasından (PNG `IEND`, JPEG `EOI`) sonra kalan **trailer** baytlarını tarayıp, içinde bilinen bir video/konteyner imzası olup olmadığını tespit eden `scripts/detect_trailer.py` script'ini yazmak.

Yaklaşım

1. Gerçek EOF'un bulunması (ham bayt taraması yerine format parsing'i)

Trailer'ı doğru bulabilmek için önce görselin **gerçek** bitiş noktasını bilmek gerekiyor. Dosyada `FF D9` veya `IEND` baytlarını ham olarak aramak yeterli değil, çünkü:

- **PNG:** `IEND` chunk'ının type alanı sabit baytlardır ama güvenilir offset'i bulmak için chunk zincirinin baştan (`length + type + data + CRC`) takip edilmesi gerekir — `find_png_end()` bunu yapar, `IEND` chunk'ının CRC'sinin bittiği yeri döndürür.
- **JPEG:** `FF D9` baytları, JPEG'de gömülü bir EXIF thumbnail'ının kendi `EOI` 'si olarak da karşımıza çıkabilir; ayrıca entropy-coded tarama verisinin içinde `FF` baytları bayt-stuffing (`FF 00`) ile korunur. Bu yüzden `find_jpeg_end()` marker'ları baştan sona **gerçek bir JPEG parser'ı gibi** takip eder: `SOS` sonrası entropy verisini, restart marker'larını (`RSTn`) atlayarak tarar ve dosyanın **son/gerçek** `EOI` 'sini bulur (ilk rastgele `FF D9` baytını değil).

Bu yaklaşım Gün 2'deki `docs/format-notlari.md` 'de not edilen riski ("JPEG'de EOI sonrası veri her zaman gizli veri anlamına gelmez") ele almak için önemliydi — ham bir `data.find(b"\xff\xd9")` araması gömülü thumbnail'lerde yanlış (çok erken) bir offset döndürebilirdi.

2. Trailer içinde video imzası taraması

Görsel bitiş offset'inden sonraki baytlar (`trailer`) çıkarılıp, bilinen konteyner imzaları aranıyor:

İmza	Aranan baytlar	Box/dosya başlangıcına göre offset
<code>mp4/ftyp</code>	<code>ftyp</code>	-4 (size alanı imzadan önce gelir)
<code>mp4/moov</code>	<code>moov</code>	-4
<code>mp4/mdat</code>	<code>mdat</code>	-4

İmza	Aranan baytlar	Box/dosya başlangıcına göre offset
avi/riff	RIFF	0
webm-mkv/ebml	1A 45 DF A3	0

3. Yanlış pozitifin önlenmesi

İki katmanlı koruma uygulandı: - **Minimum trailer boyutu eşiği** (`MIN_TRAILER_SIZE` = 16 bayt) — çok küçük trailer'lar (birkaç baytlık zararsız padding) sinyal taramasına bile sokulmuyor. - **Bilinen imza zorunluluğu** — eşik aşılsa bile, tanınan bir video/konteyner imzası bulunamazsa `polyglot_status: false` kalıyor ve trailer "muhtemelen zararsız padding/metadata" olarak raporlanıyor.

Script: `scripts/detect_trailer.py`

```
python scripts/detect_trailer.py --file <görsel>
python scripts/detect_trailer.py --file <görsel> --json
```

Çıktı alanları: `file`, `file_size`, `image_format`, `image_end_offset`, `trailer_size`, `polyglot_status`, `detected_signature`, `hidden_video_offset`, `analysis_summary`. Alan adları (`polyglot_status`, `analysis_summary`) Gün 14'te tasarlanacak `AnalyzeResponse` şemasıyla uyumlu olacak şekilde seçildi.

Test Sonuçları

Dosya	Beklenen offset (Gün 3 raporu)	Script sonucu	Polyglot mu?
<code>samples/polyglot_png.png</code>	217 (0xD9)	217 (0xD9), imza <code>mp4/ftyp</code>	EVET
<code>samples/polyglot_jpg.jpg</code>	1371 (0x55B)	1371 (0x55B), imza <code>mp4/ftyp</code>	EVET
<code>samples/sample.png</code> (temiz)	—	trailer 0 bayt	hayır
<code>samples/sample.jpg</code> (temiz)	—	trailer 0 bayt	hayır
<code>sample.jpg</code> + 5 bayt <code>0x00</code> padding	—	trailer 5 bayt, imza yok	hayır (false-positive yok)

Polyglot dosyalarda tespit edilen gizli video offset'i, Gün 3'te `make_polyglot.py` 'nin bildirdiği offset'lerle **birebir örtüşüyor** (217 ve 1371) — bu da hem üretim hem tespit tarafının tutarlı olduğunu doğruluyor.

Kabul Kriterleri — Durum

- [x] Gün 3'te üretilen polyglot dosyada gizli video başlangıç offset'i doğru tespit ediliyor
- [x] Trailer içermeyen temiz bir görselde script "polyglot değil" sonucunu doğru veriyor (false-positive yok)
- [x] (Ek test) Küçük, zararsız padding içeren dosyada da false-positive üretmiyor

Notlar / Riskler

- MP4 imza taraması yalnızca `ftyp / moov / mdat` type alanlarını arıyor; `box size` alanının aritmetik olarak doğrulanması (sayaç bazlı box gezinme) kapsam dışı bırakıldı — mevcut tehdit modelinde (trailer-append polyglot) bu yeterli, çünkü box gerçekten trailer'ın başında bulunuyor.
- JPEG parser'ı progressive JPEG'lerdeki çoklu `SOS` segmentlerini de (entropy verisi + restart marker'ları atlayarak) doğru şekilde ele alacak şekilde yazıldı, ancak bu proje kapsamında yalnızca baseline JPEG örnekleriyle test edildi.
- `scripts/detect_trailer.py`, format tespiti için Gün 3'teki `make_polyglot.py` içindeki `detect_image_format` 'ı içe aktararak tekrar kullanıyor (kod tekrarını önlemek için).